Analysis of Massive Datasets

http://www.fer.hr/predmet/avsp

Izv. prof. dr. sc. Goran Delač

Izv. prof. dr. sc. Marin Šilić

Izv. prof. dr. sc. Klemo Vladimir

Doc. dr. sc. Adrian Satja Kurdija

Faculty of Electrical Engineering and Computing
Consumer Computing Laboratory

Matrix Factorization (and Beyond) in Recommender Systems

Doc. dr. sc. Adrian Satja Kurdija

Outline

- **Introduction**
- **Matrix factorization**
  - Basic model (SGD, ALS)
  - Improving the model
    - biases
    - implicit data
    - temporal dynamics
    - confidence weights
- **Advanced models:**
  - eALS
  - Metric factorization
  - Autoencoders
  - Neural Collaborative Filtering (NCF)
  - Feature-based models (DLRM)
  - Graph-based recommendation (LightGCN)
  - Sequential recommendation (BERT4Rec)

Copyright © 2026 CCL: Analysis of massive data sets

Introduction

- **Basic strategies in recommender systems:**
  - Content filtering
  - Collaborative filtering
- **Content filtering:**
  - Recommends based on profiles for each user or product
  - For movies: genre, director, actors, box office popularity...
  - For users: demographic information, age category, questionnaires...
  - Music Genome Project (used in *Pandora.com*): each song is scored on hundreds of characteristics by a music analyst – *song genes*

Copyright © 2026 CCL: Analysis of massive data sets

Collaborative filtering

- **Collaborative filtering: rely only on past user behaviour – no need for external data**
- **Two primary areas:**
  - Neighborhood methods
    - Centered on computing user (or item) relationships
    - Indentifiy like-minded users who can complement each other's ratings
    - Indentifiy similar items which can complement each other's ratings
  - Latent factor models
    - Try to characterize each user and item with several (e.g. 20) factors inferred from the rating patterns
    - Such factors replace the human-created attributes (such as song genes)

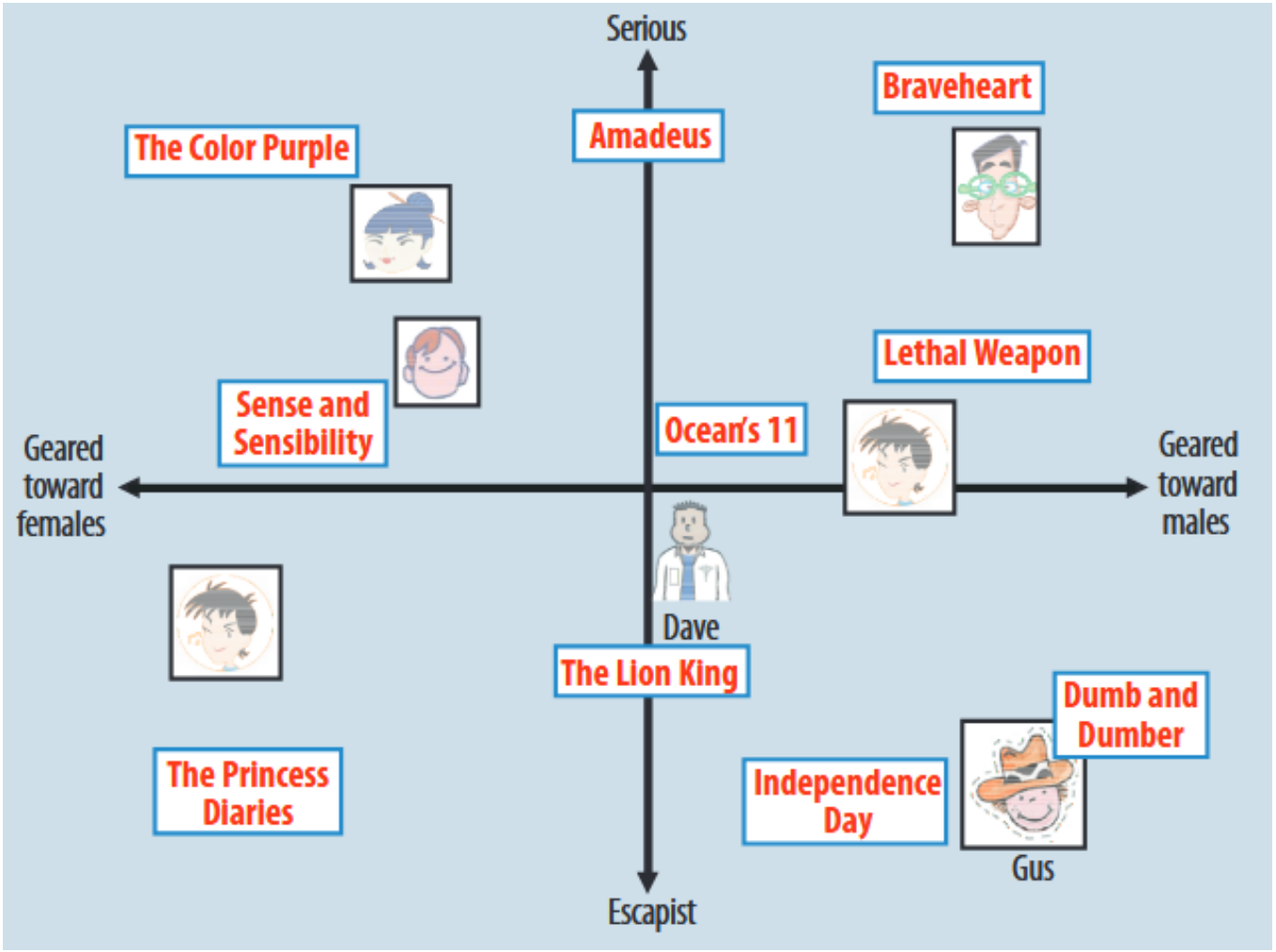
Copyright © 2026 CCL: Analysis of massive data sets

Latent factor models

- **Example for movies:**
  - „Obvious” dimensions:
    - One factor measures comedy vs. drama
    - One factor measures amount of action
    - One factor measures orientation to children
  - Less well-defined dimensions:
    - One factor measures depth of characted development
    - One factor measures quirkiness
    - Other factors measure uninterpretable dimensions
- **For users:**
  - Each factor measures how much the user likes the corresponding movie factor

Copyright © 2026 CCL: Analysis of massive data sets

Latent factor models



- Illustration from [1]: two latent factors (dimensions)

Copyright © 2026 CCL: Analysis of massive data sets

Matrix Factorization

- **Basic model:**
  - Let K be the dimensionality of the latent factor space (number of features)
  - Each item i is associated with a K-dimensional vector q_i
    - Elements of q_i measure the extent to which the item possesses the corresponding features
  - Each user u is associated with a K-dimensional vector p_u
    - Elements of p_u measure the extent of interest the user has in the corresponding features
  - Rating estimate for user u and item i is the dot product ${p_u}^T q_i$
  - Task: infer p_u and q_i from known ratings, so that ${p_u}^T q_i \approx R_{ui}$

Copyright © 2026 CCL: Analysis of massive data sets

Matrix Factorization

- **Example:**
  - K = 3; latent film features correspond to amount of action, amount of romance and acting quality
  - Film *i* has 3.4 *action*, 1.5 *romance* and 1.9 *acting*
    - q_i = [3.4, 1.5, 1.9]
  - User *u* has interest of 0.5 for *action*, -1.2 for *romance* and 2.1 for *acting*
    - p_u = [0.5, -1.2, 2.1]
  - Rating estimate for user *u* and film *i*:

$$R_{ui} \approx {p_u}^T q_i = [3.4, 1.5, 1.9]^T\,[0.5, -1.2, 2.1] =$$
$$= 3.4 * 0.5 + 1.5 * (-1.2) + 1.9 * 2.1$$
$$= 3.89$$

Copyright © 2026 CCL: Analysis of massive data sets

$R \approx PQ$

- **The rating matrix R:**

	Avatar	LOTR	Titanic	Pirates
Alice	R_{11}	R_{12}	R_{13}	R_{14}
Bob	R_{21}	R_{22}	R_{23}	R_{24}
Carol	R_{31}	R_{32}	R_{33}	R_{34}
David	R_{41}	R_{42}	R_{43}	R_{44}

- **...can be estimated as a product of *P* = [rows p_u] and *Q* = [columns q_i]:**

		User factor 1	User factor 2	User factor 3
(p_1)	Alice	P_{11}	P_{12}	P_{13}
(p_2)	Bob	P_{21}	P_{22}	P_{23}
(p_3)	Carol	P_{31}	P_{32}	P_{33}
(p_4)	David	P_{41}	P_{42}	P_{43}

$\bullet$

	Avatar	LOTR	Titanic	Pirates
Item factor 1	Q_{11}	Q_{12}	Q_{13}	Q_{14}
Item factor 2	Q_{21}	Q_{22}	Q_{23}	Q_{24}
Item factor 3	Q_{31}	Q_{32}	Q_{33}	Q_{34}
	(q_1)	(q_2)	(q_3)	(q_4)

Copyright © 2026 CCL: Analysis of massive data sets

This is machine learning!

- **What is our goal?**
  - **Objective function**

    Let $\omega = \{(u, i) : \text{user } u \text{ has rated item } i\}$.

    $$L(P, Q) = \sum_{(u,i)\in\omega} (R_{u,i} - p_u^T q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2)$$

    - We look for *P* and *Q* such that the „loss" *L(P, Q)* is **minimized**
    - The first term means that "$|R_{ui} - p_u{}^T q_i|$ should be small", i.e., $p_u{}^T q_i \approx R_{ui}$
    - The second term is to prevent **overfitting**:
      - We penalize large latent factors – otherwise, a too general model would specialize for the training data only
    - λ is a hyperparameter – **regularization factor** (e.g. 0.01)

Copyright © 2026 CCL: Analysis of massive data sets

Learning *P* and *Q*

- **Two common approaches: stochastic gradient descent and alternating least squares**
- **Both start from random *P* and *Q***
- **Stochastic gradient descent (SGD):**
  - For each training case, calculate the current $p_u{}^T q_i$ and compare it to the actual R_{ui}
  - Prediction error: $e_{ui} = (R_{ui} - p_u{}^T q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2)$
  - Each parameter x (such as p_{u1}) should be moved in the direction opposite to the gradient $\frac{\partial e}{\partial x}$
  - The move should be proportional to the **learning rate** γ (e.g. 0.05)
    - Hyperparameters are usually determined by cross-validation

Copyright © 2026 CCL: Analysis of massive data sets

Learning *P* and *Q*

- **Stochastic gradient descent (SGD):**
  - Using partial derivatives of e_{ui}, we derive the update rules for *P* and *Q*:
    - $q_i \mathrel{+}= \gamma\left((R_{ui} - {p_u}^T q_i)p_u - \lambda q_i\right)$
    - $p_u \mathrel{+}= \gamma\left((R_{ui} - {p_u}^T q_i)q_i - \lambda p_u\right)$
  - Easy to implement and fast – $O(|\mathcal{R}|K)$ per iteration
  - Repeat iterations over training set until convergence
  - Useful for big data streams – no need to store the training case (user-item interaction)

Copyright © 2026 CCL: Analysis of massive data sets

Learning *P* and *Q*

- **Alternating least squares (ALS):**
  - If we treat *P* as fixed, we can derive the optimal *Q* by solving the least squares problem, and vice versa
  - Rotate between fixing P and fixing Q, until convergence
  - Easier to parallelize: each q_i (or p_u) can be computed independently
  - Example: eALS [4]

Copyright © 2026 CCL: Analysis of massive data sets

Adding biases

- **Ratings are not explained *only* by user-item interactions:**
  - Some users give higher ratings than others (user bias)
  - Some items receive higher ratings than others (item bias)
- **A more accurate model:**
  - $R_{ui} \approx \mu + b_u + b_i + p_u^T q_i$
  - μ is the global average
  - b_u is the observed deviation of user *u* from the average
  - b_i is the observed deviation of item *i* from the average

Copyright © 2026 CCL: Analysis of massive data sets

Adding biases

- **How to incorporate biases into MF model?**
  - One way: simply learn more parameters…

$$L(P, Q, b^{(user)}, b^{(item)}) = \sum_{(u,i)\in\omega} (R_{u,i} - \mu - b_u^{(user)} - b_i^{(item)} - p_u^T q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^{(user)2} + b_i^{(item)2})$$

  - But there's a nice trick:

$b_u + b_i + puTqi$ can be written as $[b_u, 1, pu]^T[1, b_i, q_i]$.

  - So it's still $\tilde{p_u}^T \tilde{q_i}$ for "extended" $\tilde{p_u}$ and $\tilde{q_i}$.

Copyright © 2026 CCL: Analysis of massive data sets

Additional input sources

- **Cold start problem:**
  - Many users supply very few ratings
  - Predictions on their taste are difficult
- **Solution: implicit data**
  - Use behavioral feedback
  - Record any user-item interaction (such as view or click)
  - Instead of ratings, use the Boolean interaction dataset (rating or no rating, interest or no interest)
  - *Sample based learning:* sample negative instances from missing data
  - *Whole-data based learning:* use all missing data as negative instances

Copyright © 2026 CCL: Analysis of massive data sets

Temporal dynamics

- **The problem of static models:**
  - items' popularities change over time,
  - users' preferences change over time.
- **Solution: more parameters**
  - Our current parameters are actually functions of time:

$$R_{ui}(t) \approx \mu + b_u(t) + b_i(t) + p_u^T(t) q_i(t)$$

  - These functions are modelled by linear/spline formulas containing period-related parameters
    - e.g. item biases for each month
  - Result: more variables, more complicated loss function and SGD – see paper [2].

Copyright © 2026 CCL: Analysis of massive data sets

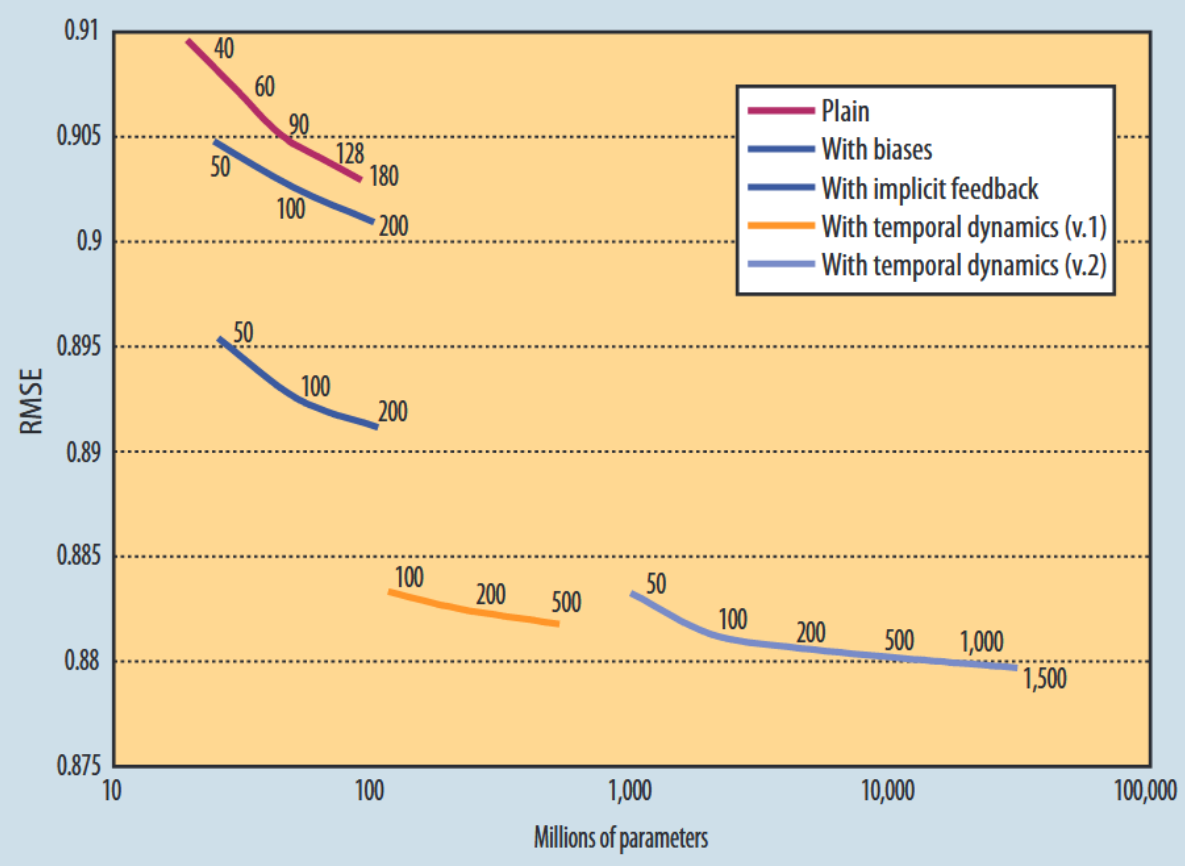
Varying confidence of inputs

- **Not all observed ratings deserve the same weight or confidence – see paper [3]**
  - Examples:
    - Short-term advertising bias
    - Dishonest/adversarial users
- **This is especially important for implicit data**
  - Confidence can stem from available numerical values:
    - Duration/frequency of actions
- **MF can incorporate confidence levels:**
  - Assume that r_{ui} has a weight of c_{ui}
  - Then loss is redefined: $$\sum_{(u,i)\in\omega} c_{ui}(R_{u,i} - \mu - b_u^{(user)} - b_i^{(item)} - p_u^T q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^{(user)2} + b_i^{(item)2})$$

Copyright © 2026 CCL: Analysis of massive data sets

Netflix prize competition

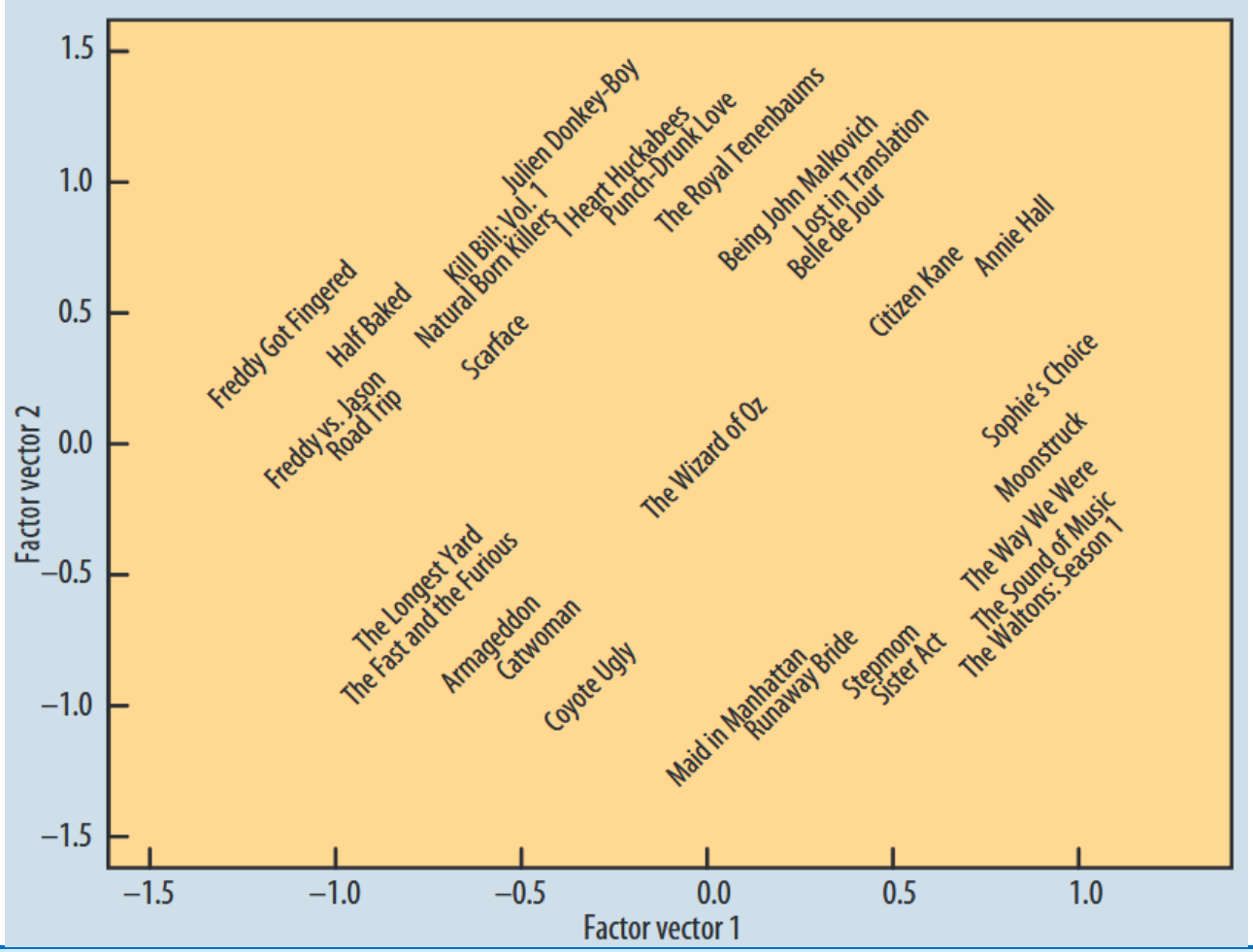
- MF was developed as the winning entry (2007)
- Accuracies of individual models [1]:



Copyright © 2026 CCL: Analysis of massive data sets

Netflix prize competition

- **Illustration – first two latent factors of selected movies:**
  - x-axis is „lowbrow vs. serius”, y-axis is „mainstream vs. indie” [1]:



Copyright © 2026 CCL: Analysis of massive data sets

Model evaluation example

- **From eALS paper [4]**
- ***Yelp* and *Amazon Movies* review datasets**
- **Treat each review as 1 and rest of the data as 0**
- **Filter out users and items with less than 10 interactions**
- ***Leave-one-out* protocol:**
  - Latest interaction of each user is held out for prediction
  - Model is trained on remaining data
- **Online protocol:**
  - Sort the interactions in chronological order
  - Train on first 90%, test on last 10% with incremental updates

Copyright © 2026 CCL: Analysis of massive data sets

Model evaluation example

- **For each test interaction:**
  - Randomly sample 100 items not interacted by the user
  - Rank the test item among the 100 items
- **Accuracy measures:**
  - **Hit ratio (HR):** how often is the test item present in the top 10 list (0% – 100%)?
  - **Normalized Discounted Cumulative Gain (NDCG):** higher position (top 1 – top 100) gets a higher score (0 – 1)

Copyright © 2026 CCL: Analysis of massive data sets

Metric factorization [6]

- **Learn K-dimensional positions of users and items such that:**

  rating(*user*, *item*) ≈ maxRating – **distance**(*user*, *item*)

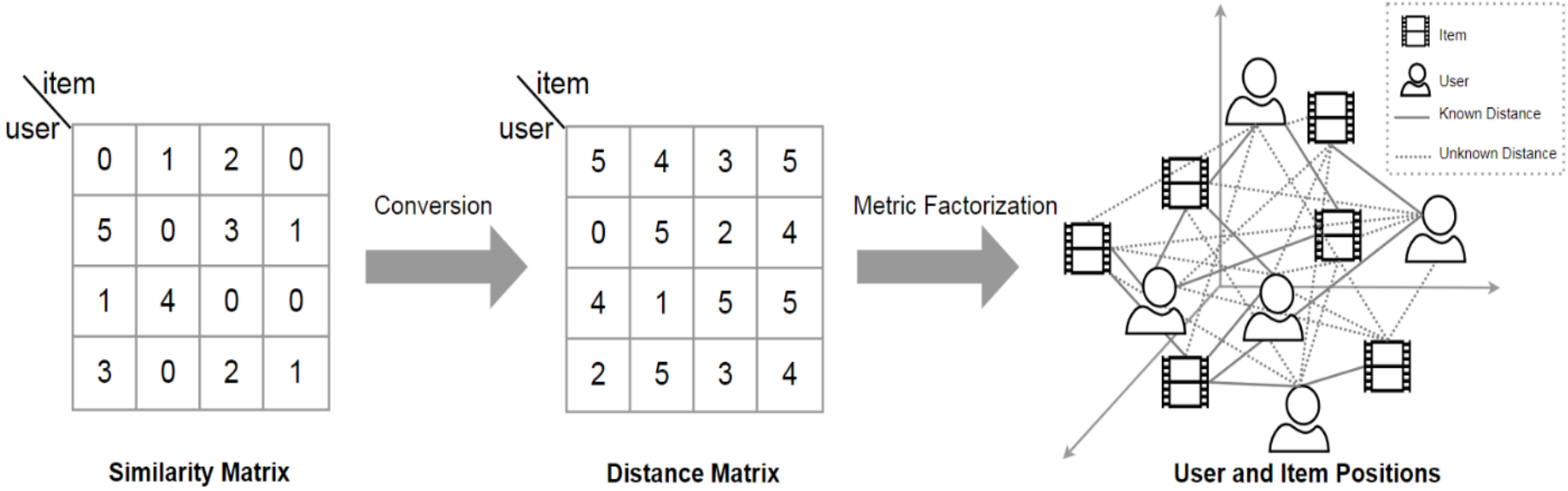


Figure 2: A simplified illustration of Metric Factorization. It mainly has two steps: First, convert similarity matrix (rating matrix or interaction matrix) to distance matrix via equation 2. Second, factorize the distance matrix into user and item positions in a low-dimensional space. In the last graph, solid line represents known distances and dash line indicates unknown distances. Once the positions of users & items are learned, unknown distances can be easily derived for making recommendation.

Copyright © 2026 CCL: Analysis of massive data sets

Using autoencoders

- **Finds a low-dimensional representation of user ratings vector $x \in R^n$, where n = #items**
  - $encode(x) : R^n \rightarrow R^k$
  - $decode(x) : R^k \rightarrow R^n$
  - $x \approx x' := decode(encode(x))$
- **Input vector x is *sparse*, while decoder output $x' \in R^n$ is *dense* and contains rating predictions for all items**
- ***encode* example: $e = f(Wx + b)$**
  - e is a k-dimensional latent vector
  - W is the weight matrix of size $k \times n$
  - b is the bias vector of size k
  - f is some non-linear activation function (e.g. SELU)

Copyright © 2026 CCL: Analysis of massive data sets

DeepRec autoencoder model [7]

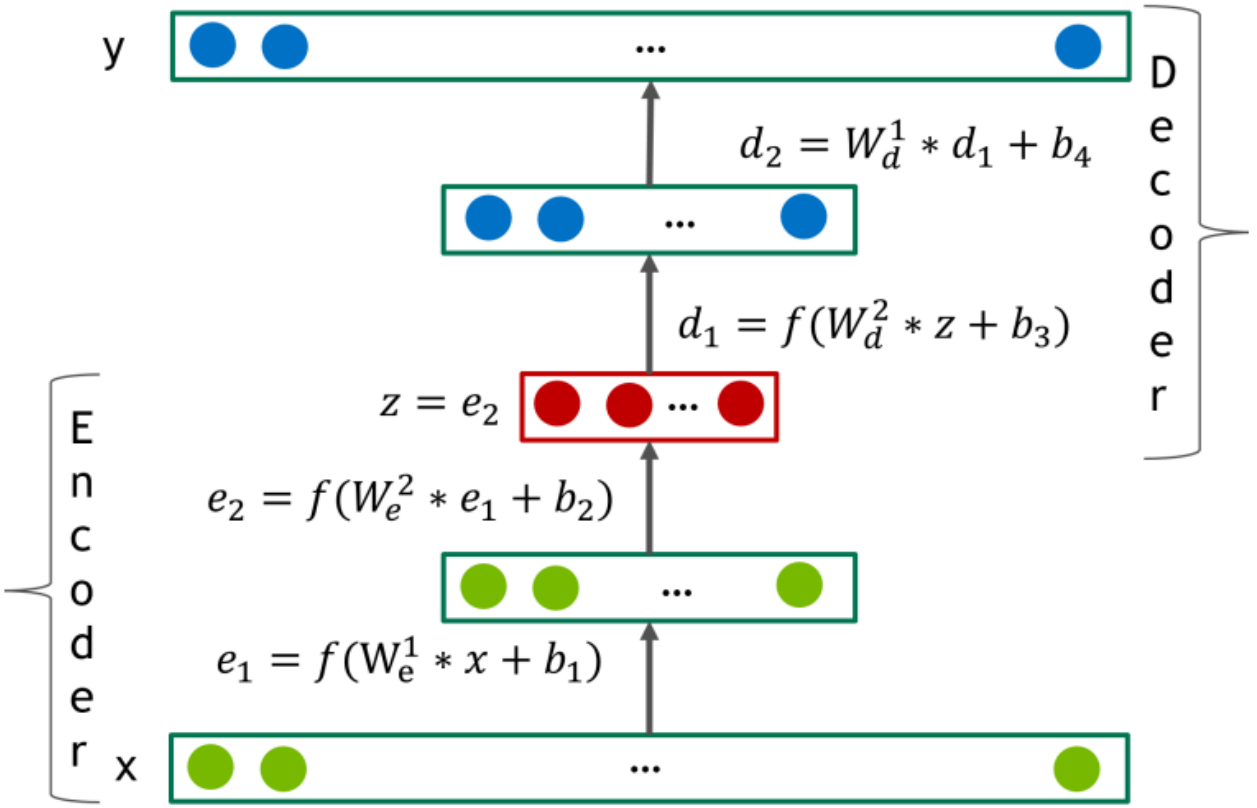


Figure 1: AutoEncoder consists of two neural networks, encoder and decoder, fused together on the "representation" layer z. Encoder has 2 layers e_1 and e_2 and decoder has 2 layers d_1 and d_2. Dropout may be applied to coding layer z.

Copyright © 2026 CCL: Analysis of massive data sets

Neural Collaborative Filtering

- **He et al. [5] have noticed that an inner product $p_u{}^T q_i$ is limiting**
- **A model which predicts with a more general function $f(p_u, qi)$ could be more powerful**
- **The first idea:**
  - Learn constant weights for the inner product summands
  - Apply some non-linear function to the result (e.g. sigmoid):

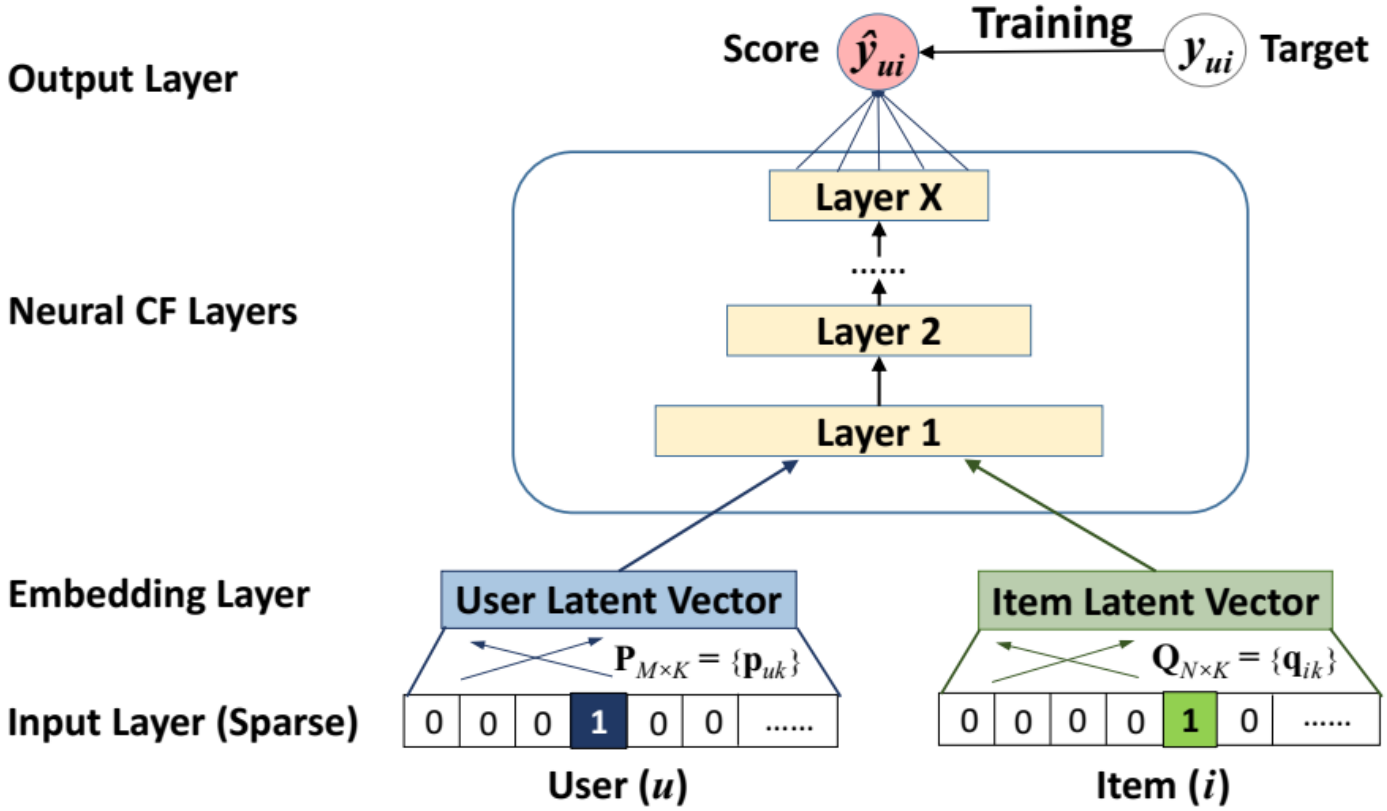
$$\widetilde{r_{ui}} = \sigma(\boldsymbol{h}^T(p_u \cdot q_i))$$

- **This is Generalized Matrix Factorization (GMF)**
  - Standard MF is a special case ($h = [1, 1, \ldots, 1]$ and $\sigma(x) = x$)

Copyright © 2026 CCL: Analysis of massive data sets

Neural Collaborative Filtering

- Second idea: the function can also be a neural network:



- This is Multi-Layer Perceptron (MLP)

Copyright © 2026 CCL: Analysis of massive data sets

Neural Collaborative Filtering

- Combining GMF and MLP we get the NeuMF [5]:

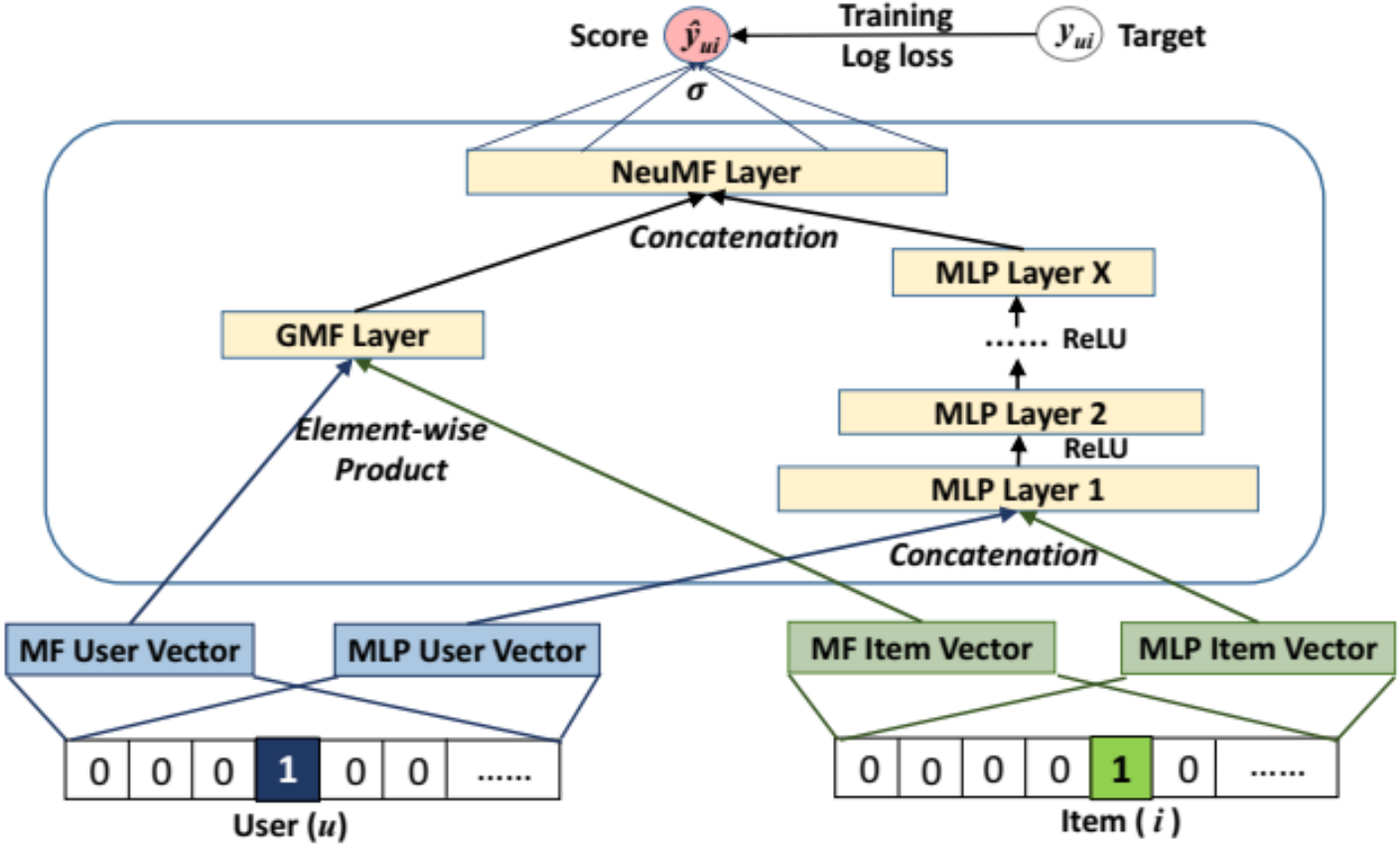


Figure 3: Neural matrix factorization model

Copyright © 2026 CCL: Analysis of massive data sets

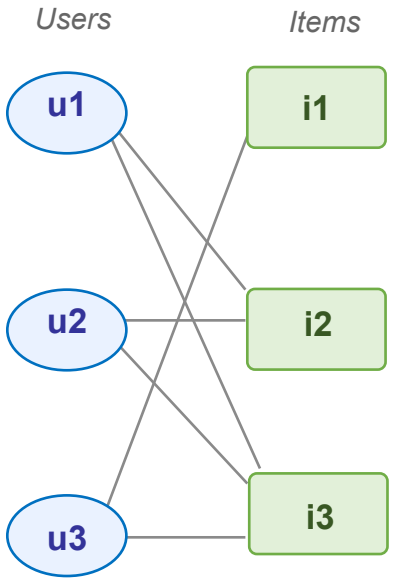
Feature-based scoring models

- **We can generalize NCF by adding embeddings for many features: user attributes, item attributes, device, country, time, session, query, etc.**
  - This yields models such as Deep Learning Recommendation Model (DLRM) [10]
  - They predict score for a *(user, item, context)* tuple
- **Large Language Models (LLMs) can encode unstructured text such as descriptions, tags, reviews, and comments into dense feature vectors**
- **LLM-derived representations are used as additional embeddings for scoring, cold start, and explanation**

Copyright © 2026 CCL: Analysis of massive data sets

Graph-based recommendation

- CF inherently models a user-item bipartite graph
- Graph Neural Networks (GNNs) propagate embeddings along the graph edges to obtain graph-aware embeddings
- LightGCN [8]: user representation is enriched by aggregating the representations of the neighboring items (and vice versa) in 2-4 steps (hops)
- Final user and item embeddings are obtained by combining the embeddings from all hops
- Rating prediction is computed as the dot product of the final user and item embeddings: $e_u^\top e_i$



MF:
0 hops

LightGCN:
0-hop
+ 1-hop
+ 2-hop

Embedding propagation:

$$e_u^{(k+1)} = \sum_{i \in \mathcal{N}(u)} \frac{1}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(i)|}} e_i^{(k)}$$

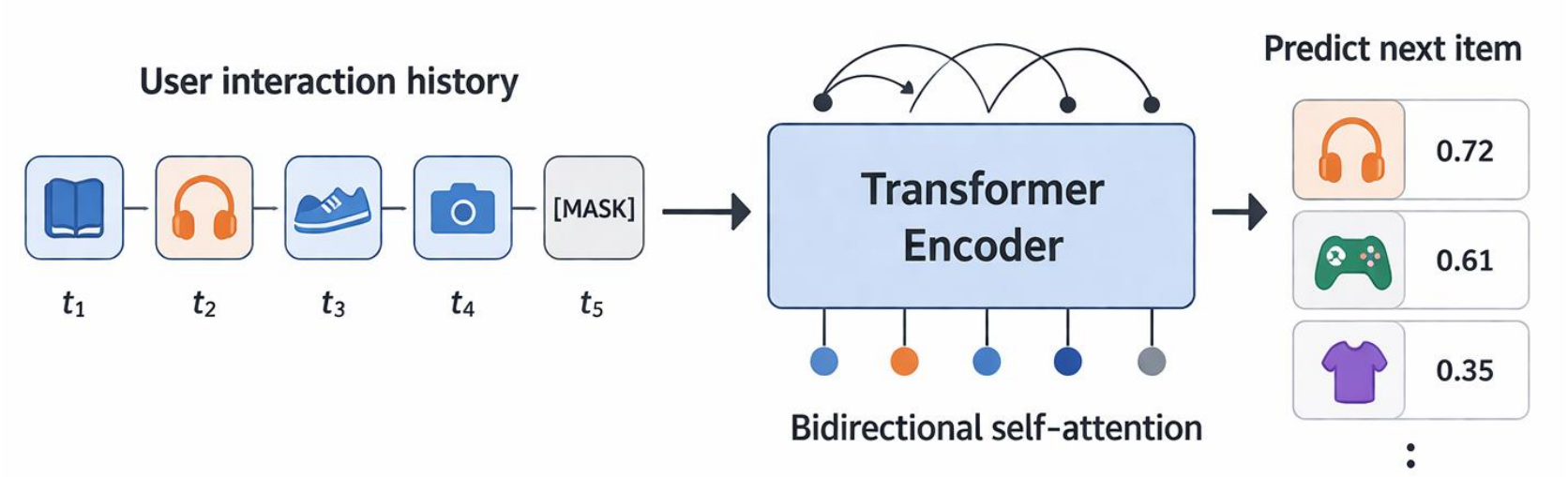
Final embedding: hops average

$$e_u = \sum_{k=0}^{K} \alpha_k e_u^{(k)}$$

Copyright © 2026 CCL: Analysis of massive data sets

Sequential recommendation: BERT4Rec [9]

- Real systems must model short-term intent, recency, current context, and order effects
- Sequential recommendation models the interaction history as a sequence that represents the evolving user behavior better than static latent factors
- Useful for fast-changing preferences: feeds, sessions, and shopping journeys



Copyright © 2026 CCL: Analysis of massive data sets

References

1. Matrix factorization techniques for recommender systems - Y Koren, R Bell, C Volinsky - *Computer*, 2009
2. Collaborative Filtering with Temporal Dynamics – Y Koren - *Communications of the ACM*, 2010
3. Collaborative Filtering for Implicit Feedback Datasets – Y Hu, Y Koren, C Volinsky - *ICDM* '08
4. Fast Matrix Factorization for Online Recommendation with Implicit Feedback - X He, H Zhang, MY Kan, TS Chua - *SIGIR* 2016
5. Neural collaborative filtering - X He, L Liao, H Zhang, L Nie, X Hu - *26th International Conference on World Wide Web* (WWW '17)
6. Metric Factorization: Recommendation beyond Matrix Factorization – Zhang, S., Yao, L., Tay, Y., Xu, X., Zhang, X., & Zhu, L. – ArXiv (2018)

Copyright © 2026 CCL: Analysis of massive data sets

References

7. Training Deep AutoEncoders for Collaborative Filtering - Oleksii Kuchaiev, Boris Ginsburg – ArXiv (2017)
8. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation – He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. - SIGIR 2020.
9. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformers – Fei Sun, Jianping Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, Peng Jiang – CIKM 2019.
10. Deep Learning Recommendation Model for Personalization and Recommendation Systems – M. Naumov et al. – ArXiv (2019)

Copyright © 2026 CCL: Analysis of massive data sets